1. Introduction

Kotlin is a modern, statically-typed programming language that runs on the Java Virtual Machine (JVM). It is fully interoperable with Java and offers several advantages:

- **Conciseness**: Less boilerplate code compared to Java
- **Null Safety**: Built-in null safety mechanism
- **Functional Programming**: First-class support for functional programming paradigms
- **Extension Functions**: Extend classes without inheritance
- **Coroutines**: Lightweight concurrency model
- **Multi-platform**: Can compile to JVM, JavaScript, and native code

Getting Started

To use Kotlin, you need:

- Java Development Kit (JDK) 8 or later
- Kotlin compiler (installed via Gradle, Maven, or standalone)

Hello World

```
fun main() {
    println("Hello, World!")
}
```

---

2. Basic Syntax

Statements and Expressions

In Kotlin, almost everything is an expression (returns a value):

```
// Statement style
val x = 5

// Expression style
val y = if (x > 0) "positive" else "non-positive"
```

Comments

```
// Single-line comment

/*
Multi-line comment
Can span multiple lines
*/
```

```kotlin
/**
 * Documentation comment
 * Used for generating API documentation
 */
fun exampleFunction() {}
```

Statement Terminators

Kotlin uses semicolons optionally. In most cases, you don't need them:

```kotlin
val x = 10
val y = 20  // Semicolon is optional

val z = 30; val w = 40  // Only needed to separate multiple statements on same line
```

---

3. Variables and Data Types

Variable Declaration

`val` - Immutable (Read-only)

```kotlin
val name = "Alice"
// name = "Bob"  // Error: Val cannot be reassigned
```

`var` - Mutable

```kotlin
var age = 25
age = 26  // OK
```

Type Inference

Kotlin infers types automatically:

```kotlin
val number = 42          // Type inferred as Int
val decimal = 3.14       // Type inferred as Double
val isActive = true      // Type inferred as Boolean
```

Explicit Type Declaration

```kotlin
val name: String = "Alice"
val age: Int = 25
val balance: Double = 1000.50
val isActive: Boolean = true
```

Basic Data Types

Integer Types

```kotlin
val byte: Byte = 127              // 8-bit
val short: Short = 32767          // 16-bit
val int: Int = 2147483647         // 32-bit (default for integers)
val long: Long = 9223372036854775807L  // 64-bit
```

Floating-Point Types

```kotlin
val float: Float = 3.14f          // 32-bit
val double: Double = 3.14159      // 64-bit (default for decimals)
```

Character and Boolean

```kotlin
val letter: Char = 'A'
val isKotlin: Boolean = true
```

Strings

```kotlin
val name: String = "Kotlin"

// String templates
val age = 25
val greeting = "Hello, my name is $name and I'm $age years old"

// Expression in string template
val sum = "${10 + 20}"

// Raw string with multi-line
val multiLine = """
    Line 1
    Line 2
    Line 3
""".trimIndent()
```

Type Conversion

Kotlin requires explicit conversion:

```
val number = 10
val decimal = number.toDouble()
val text = number.toString()
val intValue = "42".toInt()

// Converting between numeric types
val x: Int = 5
val y: Long = x.toLong()
val z: Double = x.toDouble()
```

Constants

```
const val MAX_SIZE = 100  // Compile-time constant
val maxSize = 100         // Runtime constant
```

---

4. Operators

Arithmetic Operators

```
val a = 10
val b = 3

val sum = a + b          // 13
val difference = a - b   // 7
val product = a * b      // 30
val quotient = a / b     // 3 (integer division)
val remainder = a % b    // 1
```

Assignment Operators

```
var x = 5
x += 3   // x = x + 3 = 8
x -= 2   // x = x - 2 = 6
x *= 2   // x = x * 2 = 12
x /= 4   // x = x / 4 = 3
x %= 2   // x = x % 2 = 1
```

Comparison Operators

```
val a = 5
val b = 10

a == b       // false (equality)
a != b       // true (inequality)
a < b        // true
a > b        // false
a <= b       // true
a >= b       // false
```

Logical Operators

```
val isAdult = true
val hasLicense = false

isAdult && hasLicense    // false (AND)
isAdult || hasLicense    // true (OR)
!isAdult                 // false (NOT)
```

Bitwise Operators

```
val a = 5      // 0101
val b = 3      // 0011

a and b        // 0001 = 1 (bitwise AND)
a or b         // 0111 = 7 (bitwise OR)
a xor b        // 0110 = 6 (bitwise XOR)
a.inv()        // inverts bits
a shl 1        // left shift: 1010 = 10
a shr 1        // right shift: 0010 = 2
```

Increment and Decrement

```
var counter = 5
counter++      // Post-increment: returns 5, then increments
counter--      // Post-decrement: returns 6, then decrements
++counter      // Pre-increment: increments, then returns
--counter      // Pre-decrement: decrements, then returns
```

Range Operators

```
val range = 1..5        // 1, 2, 3, 4, 5 (inclusive)
val range2 = 1..<5      // 1, 2, 3, 4 (exclusive end)
```

```
val range3 = 5 downTo 1 // 5, 4, 3, 2, 1 (descending)
val range4 = 1..10 step 2 // 1, 3, 5, 7, 9
```

in Operator

```
val numbers = 1..10
5 in numbers          // true
15 in numbers         // false
5 !in numbers         // false
```

---

5. Control Flow

if Expression

```
val age = 25

if (age >= 18) {
    println("Adult")
}

val status = if (age >= 18) "Adult" else "Minor"

val category = when {
    age < 13 -> "Child"
    age < 18 -> "Teenager"
    else -> "Adult"
}
```

when Expression

```
val number = 2

when (number) {
    1 -> println("One")
    2 -> println("Two")
    3 -> println("Three")
    else -> println("Other")
}

// Multiple values per branch
when (number) {
    1, 2, 3 -> println("Small")
    4, 5, 6 -> println("Medium")
    else -> println("Large")
}
```

```
// Range matching
when (number) {
    in 1..10 -> println("Between 1 and 10")
    in 11..20 -> println("Between 11 and 20")
    else -> println("Outside range")
}

// Type checking
fun demo(x: Any) {
    when (x) {
        is String -> println("String: ${x.length}")
        is Int -> println("Number: $x")
        else -> println("Unknown type")
    }
}
```

for Loop

```
// Range iteration
for (i in 1..5) {
    println(i)
}

// Reverse iteration
for (i in 5 downTo 1) {
    println(i)
}

// With step
for (i in 1..10 step 2) {
    println(i)
}

// Collection iteration
val fruits = listOf("Apple", "Banana", "Cherry")
for (fruit in fruits) {
    println(fruit)
}

// Index iteration
for (index in fruits.indices) {
    println("$index: ${fruits[index]}")
}

// Destructuring in loops
val pairs = listOf(1 to "a", 2 to "b", 3 to "c")
for ((number, letter) in pairs) {
    println("$number -> $letter")
}
```

while Loop

```
var count = 0
while (count < 5) {
    println(count)
    count++
}

// do-while loop
var counter = 0
do {
    println(counter)
    counter++
} while (counter < 5)
```

break and continue

```
for (i in 1..10) {
    if (i == 5) break      // Exit loop
    if (i == 3) continue   // Skip to next iteration
    println(i)
}

// Labeled break/continue
outer@ for (i in 1..3) {
    for (j in 1..3) {
        if (i == 2 && j == 2) break@outer  // Break outer loop
        println("$i, $j")
    }
}
```

---

6. Functions

Basic Function Definition

```
fun greet(name: String): String {
    return "Hello, $name!"
}

// Usage
println(greet("Alice"))
```

Single Expression Functions

```
fun greet(name: String): String = "Hello, $name!"

// Type can be inferred
fun greet(name: String) = "Hello, $name!"
```

Default Parameters

```
fun greet(name: String, greeting: String = "Hello"): String {
    return "$greeting, $name!"
}

greet("Alice")                    // "Hello, Alice!"
greet("Bob", "Hi")                // "Hi, Bob!"
greet(greeting = "Hey", name = "Charlie")  // "Hey, Charlie!"
```

Variable Number of Arguments (varargs)

```
fun sumNumbers(vararg numbers: Int): Int {
    return numbers.sum()
}

sumNumbers(1, 2, 3, 4, 5)  // 15

// Spread operator
val numbers = intArrayOf(1, 2, 3)
sumNumbers(*numbers)  // 6
```

Named Arguments

```
fun printInfo(name: String, age: Int, city: String) {
    println("Name: $name, Age: $age, City: $city")
}

printInfo(name = "Alice", age = 25, city = "NYC")
printInfo("Alice", city = "NYC", age = 25)  // Arguments can be reordered
with names
```

Return Type

```
fun noReturn(): Unit {
    println("This function returns Unit")
}
```

```
// Unit is implicit, can be omitted
fun noReturn() {
    println("This function returns Unit")
}

// Nothing type - function never returns
fun throwError(): Nothing {
    throw IllegalArgumentException("Error!")
}
```

Local Functions

```
fun outer() {
    fun inner() {
        println("Inner function")
    }
    inner()  // Can call inner function
}

// outer()
// inner()  // Error: inner is not accessible
```

Tail Recursive Functions

```
tailrec fun factorial(n: Int, accumulator: Int = 1): Int {
    return if (n <= 1) accumulator else factorial(n - 1, n * accumulator)
}

// Optimized to iterative code
println(factorial(5))  // 120
```

Function Overloading

```
fun display(value: Int) = println("Int: $value")
fun display(value: String) = println("String: $value")
fun display(value: Double) = println("Double: $value")

display(10)          // Int: 10
display("Hello")     // String: Hello
display(3.14)        // Double: 3.14
```

Infix Functions

```kotlin
infix fun Int.times(str: String) = str.repeat(this)

3 times "Kotlin"  // "KotlinKotlinKotlin"
```

---

7. Nullability

Nullable Types

By default, variables cannot be null:

```kotlin
val name: String = null  // Compilation error

// Make it nullable with ?
val name: String? = null  // OK
val name: String? = "Alice"  // OK
```

Safe Call Operator (?)

```kotlin
val name: String? = "Alice"
val length = name?.length  // Returns null if name is null, otherwise
length

val city: String? = null
val length2 = city?.length  // Returns null
```

Elvis Operator (?😃

```kotlin
val name: String? = null
val displayName = name ?: "Unknown"  // "Unknown"

val name2: String? = "Alice"
val displayName2 = name2 ?: "Unknown"  // "Alice"

// Chain Elvis operators
val result = value1 ?: value2 ?: value3 ?: "default"
```

Not-null Assertion (!!)

```kotlin
val name: String? = "Alice"
val length = name!!.length  // OK, crashes if name is null
```

```
val city: String? = null
val length2 = city!!.length  // Throws NullPointerException
```

Safe Casting with as?

```
val obj: Any = "Hello"
val str: String? = obj as? String  // "Hello"
val str2: String? = obj as? Int    // null (safe cast, no exception)
```

Let Function

```
val name: String? = "Alice"

// Only executes if name is not null
name?.let {
    println("Name is $it and has ${it.length} characters")
}

val city: String? = null
city?.let {
    println("City: $it")  // This won't execute
}
```

Nullable Function Types

```
val callback: ((String) -> Unit)? = null

// Safe call
callback?.invoke("Hello")

// With let
callback?.let { it("Hello") }
```

---

8. Classes and Objects

Class Definition

```
class Person {
    var name: String = ""
    var age: Int = 0
}

val person = Person()
```

```
person.name = "Alice"
person.age = 25
```

Primary Constructor

```
class Person(name: String, age: Int) {
    val name: String = name
    val age: Int = age
}

// Shorthand with val/var
class Person(val name: String, var age: Int)

val person = Person("Alice", 25)
```

Properties with Custom Getters and Setters

```
class Person(val name: String, age: Int) {
    var age: Int = age
        get() = field
        set(value) {
            if (value >= 0) field = value
        }

    val isAdult: Boolean
        get() = age >= 18
}

val person = Person("Alice", 25)
println(person.isAdult)  // true
person.age = -5  // Doesn't update, validation fails
```

Lazy Initialization

```
class User(val name: String) {
    val profile: String by lazy {
        println("Loading profile for $name")
        "Profile of $name"
    }
}

val user = User("Alice")
println(user.profile)  // Loads on first access
println(user.profile)  // Uses cached value
```

Init Block

```
class Person(val name: String, age: Int) {
    var age: Int = age
        set(value) {
            if (value >= 0) field = value
        }

    init {
        println("Creating person: $name with age: $age")
    }
}

Person("Alice", 25)  // Prints: "Creating person: Alice with age: 25"
```

Secondary Constructor

```
class Person(val name: String) {
    var age: Int = 0

    constructor(name: String, age: Int) : this(name) {
        this.age = age
    }
}

val person1 = Person("Alice")
val person2 = Person("Bob", 30)
```

Methods

```
class Calculator {
    fun add(a: Int, b: Int): Int = a + b
    fun subtract(a: Int, b: Int): Int = a - b
    fun multiply(a: Int, b: Int): Int = a * b
}

val calc = Calculator()
println(calc.add(5, 3))  // 8
```

this Keyword

```
class Person(val name: String, val age: Int) {
    fun introduce() = "Hello, I'm ${this.name} and I'm ${this.age} years
old"
```

```
    fun copy(name: String = this.name, age: Int = this.age) =
        Person(name, age)
}
```

---

9. Inheritance

Basic Inheritance

```
open class Animal(val name: String) {
    open fun sound() = "Some sound"
}

class Dog(name: String) : Animal(name) {
    override fun sound() = "Woof!"
}

val dog = Dog("Buddy")
println(dog.sound())  // Woof!
```

Overriding Properties

```
open class Shape {
    open val area: Double = 0.0
}

class Circle(val radius: Double) : Shape() {
    override val area: Double = 3.14 * radius * radius
}
```

super Keyword

```
open class Animal {
    open fun sound() = "Generic sound"
}

class Dog : Animal() {
    override fun sound() = "${super.sound()} - Woof!"
}

println(Dog().sound())  // Generic sound - Woof!
```

Is-A Relationship

```
open class Vehicle
class Car : Vehicle()

val car = Car()
println(car is Car)      // true
println(car is Vehicle)  // true
println(car is String)   // false
```

---

10. Interfaces

Basic Interface

```
interface Animal {
    fun sound(): String
    val name: String
}

class Dog(override val name: String) : Animal {
    override fun sound() = "Woof!"
}

val dog: Animal = Dog("Buddy")
println(dog.sound())  // Woof!
```

Multiple Inheritance

```
interface Animal {
    fun sound(): String
}

interface Swimmer {
    fun swim(): String
}

class Duck : Animal, Swimmer {
    override fun sound() = "Quack!"
    override fun swim() = "Swimming"
}

val duck = Duck()
println(duck.sound())  // Quack!
println(duck.swim())   // Swimming
```

Default Implementation

```
interface Animal {
    fun sound(): String
    fun sleep(): String = "Zzz"
}

class Dog : Animal {
    override fun sound() = "Woof!"
    // sleep() already has implementation
}

println(Dog().sleep())  // Zzz
```

Interface with Properties

```
interface Vehicle {
    val make: String
    val model: String
    val year: Int
}

class Car(override val make: String, override val model: String, override
val year: Int) : Vehicle
```

---

11. Abstract Classes

Abstract Class Definition

```
abstract class Animal(val name: String) {
    abstract fun sound(): String
    open fun sleep() = "Sleeping..."
}

class Dog(name: String) : Animal(name) {
    override fun sound() = "Woof!"
    override fun sleep() = "Dog sleeping peacefully"
}

val dog: Animal = Dog("Buddy")
println(dog.sound())   // Woof!
println(dog.sleep())   // Dog sleeping peacefully
```

Abstract Properties

```
abstract class Vehicle {
    abstract val make: String
    abstract val model: String
}

class Car(override val make: String, override val model: String) :
Vehicle()
```

Difference Between Interface and Abstract Class

Feature	Interface	Abstract Class
State	Properties only (no backing field by default)	Can have state with backing fields
Constructor	Cannot have constructor	Can have constructor
Multiple Inheritance	Can implement multiple	Can extend only one
Access Modifiers	public by default	Can be any modifier
Methods	Can be abstract or concrete	Can be abstract or concrete

12. Collections

Lists

```
// Immutable list
val fruits = listOf("Apple", "Banana", "Cherry")
val numbers = listOf(1, 2, 3, 4, 5)

// Access
println(fruits[0])           // Apple
println(fruits.first())      // Apple
println(fruits.last())       // Cherry
println(fruits.size)         // 3

// Mutable list
val mutableFruits = mutableListOf("Apple", "Banana")
mutableFruits.add("Cherry")
mutableFruits.remove("Apple")
mutableFruits[0] = "Orange"

// ArrayList
val arrayList = arrayListOf(1, 2, 3, 4)
```

Sets

```
// Immutable set
val uniqueNumbers = setOf(1, 2, 3, 2, 1)  // Duplicates ignored
println(uniqueNumbers.size)  // 3

// Mutable set
val mutableSet = mutableSetOf(1, 2, 3)
mutableSet.add(4)
mutableSet.remove(2)

// HashSet
val hashSet = hashSetOf(1, 2, 3)
```

Maps

```
// Immutable map
val capitals = mapOf("India" to "Delhi", "USA" to "Washington")
val capitals = mapOf(Pair("India", "Delhi"), Pair("USA", "Washington"))

// Access
println(capitals["India"])    // Delhi
println(capitals.get("USA"))  // Washington

// Mutable map
val mutableMap = mutableMapOf("a" to 1, "b" to 2)
mutableMap["c"] = 3
mutableMap.remove("a")

// HashMap
val hashMap = hashMapOf("x" to 10, "y" to 20)

// Iteration
for ((country, capital) in capitals) {
    println("$country -> $capital")
}

// get with default value
println(capitals["Canada"] ?: "Unknown")
```

Collection Operations

```
val numbers = listOf(1, 2, 3, 4, 5)

// Filter
val evenNumbers = numbers.filter { it % 2 == 0 }  // [2, 4]

// Map
val doubled = numbers.map { it * 2 }  // [2, 4, 6, 8, 10]
```

```
// Find
val firstEven = numbers.find { it % 2 == 0 }  // 2

// Any, All, None
numbers.any { it > 3 }         // true
numbers.all { it > 0 }         // true
numbers.none { it > 10 }       // true

// Sum, Count, Average
numbers.sum()                  // 15
numbers.count()                // 5
numbers.average()              // 3.0

// Min, Max
numbers.minOrNull()            // 1
numbers.maxOrNull()            // 5

// Fold
numbers.fold(0) { acc, value -> acc + value }  // 15

// Reduce
numbers.reduce { acc, value -> acc + value }  // 15

// GroupBy
val grouped = listOf(1, 2, 3, 4, 5, 6)
    .groupBy { it % 2 }  // {1=[1,3,5], 0=[2,4,6]}

// Flatten
val nested = listOf(listOf(1, 2), listOf(3, 4))
nested.flatten()  // [1, 2, 3, 4]

// FlatMap
listOf(1, 2, 3).flatMap { listOf(it, it * 2) }  // [1, 2, 2, 4, 3, 6]

// Distinct
listOf(1, 2, 2, 3, 3, 3).distinct()  // [1, 2, 3]

// Take, Drop
numbers.take(3)   // [1, 2, 3]
numbers.drop(2)   // [3, 4, 5]

// TakeLast, DropLast
numbers.takeLast(2)   // [4, 5]
numbers.dropLast(2)   // [1, 2, 3]
```

Ranges as Collections

```
val range = 1..5
println(range.toList())  // [1, 2, 3, 4, 5]
```

```
println(range.first())   // 1
println(range.last())    // 5
```

---

13. Sequences

Sequences are lazy evaluated, unlike eager evaluation in collections:

```
val numbers = (1..10).asSequence()

val result = numbers
    .filter { println("Filter: $it"); it % 2 == 0 }
    .map { println("Map: $it"); it * 2 }
    .take(2)
    .toList()

// Filter, Map, Take evaluated lazily and stopped early

// Compare with List (eager):
val listResult = (1..10)
    .filter { println("Filter: $it"); it % 2 == 0 }
    .map { println("Map: $it"); it * 2 }
    .take(2)

// All filters and maps applied to entire list
```

When to Use Sequences

- Large collections: Sequences stop early, avoiding unnecessary operations
- Long chains: More efficient with multiple operations
- One-time iteration: Sequences are not cached

---

14. Lambda Expressions and Functional Programming

Lambda Syntax

```
// Simple lambda
val add = { a: Int, b: Int -> a + b }
println(add(5, 3))  // 8

// With inferred types
val multiply: (Int, Int) -> Int = { a, b -> a * b }

// Single parameter (it is implicit)
val square = { x: Int -> x * x }
val squareImplicit: (Int) -> Int = { it * it }

// No parameters
```

```kotlin
val greet = { "Hello, Kotlin!" }

// No return value (returns Unit)
val printName = { name: String -> println("Hello, $name") }
```

Using Lambdas with Collections

```kotlin
val numbers = listOf(1, 2, 3, 4, 5)

// With filter
numbers.filter { it > 2 }  // [3, 4, 5]

// With map
numbers.map { it * 2 }     // [2, 4, 6, 8, 10]

// With forEach
numbers.forEach { println(it) }

// Trailing lambda
numbers.filter { it % 2 == 0 }
// If lambda is last parameter, can be outside parentheses

// Multiple operations
numbers
    .filter { it % 2 == 0 }
    .map { it * 2 }
    .forEach { println(it) }  // Prints: 4, 8
```

Function Types

```kotlin
// Function type definition
val operation: (Int, Int) -> Int = { a, b -> a + b }

// Nullable function type
val callback: ((String) -> Unit)? = null

// Function returning function
val multiplyBy: (Int) -> (Int) -> Int = { factor ->
    { number -> number * factor }
}

val double = multiplyBy(2)
println(double(5))  // 10
```

---

15. Higher-Order Functions

Functions Taking Functions as Parameters

```
fun applyOperation(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
    return operation(a, b)
}

val result = applyOperation(5, 3) { x, y -> x + y }  // 8

// Another example
fun repeat(times: Int, action: () -> Unit) {
    repeat(times) { action() }
}

repeat(3) { println("Hello") }
```

Functions Returning Functions

```
fun multiplier(factor: Int): (Int) -> Int {
    return { number -> number * factor }
}

val double = multiplier(2)
val triple = multiplier(3)

println(double(5))  // 10
println(triple(5))  // 15
```

Inline Functions

```
inline fun <T> performAction(action: (T) -> Unit, value: T) {
    action(value)
}

// Inlining avoids object creation for function parameters
performAction({ x -> println(x) }, "Hello")

// With noinline
inline fun process(inlineFunc: (String) -> Unit, noinlineFunc: (String) ->
Unit) {
    inlineFunc("Inline")
    noinlineFunc("Non-inline")
}
```

Reified Type Parameters

```
inline fun <reified T> isInstance(obj: Any): Boolean {
    return obj is T
```

```
}

println(isInstance<String>("Hello"))  // true
println(isInstance<Int>(42))          // true

// Access generic type at runtime
inline fun <reified T> parseJson(json: String): T {
    // Can use T at runtime (only with inline functions)
    return when (T::class) {
        String::class -> json as T
        Int::class -> json.toInt() as T
        else -> throw IllegalArgumentException()
    }
}
```

---

16. Scope Functions

let

Executes block and returns result. Null-safe with `?.let`:

```
val name: String? = "Alice"

name?.let {
    println("Name: $it")
    println("Length: ${it.length}")
}

// Returns value from lambda
val result = name?.let { it.uppercase() } ?: "UNKNOWN"
println(result)  // ALICE
```

run

Executes block and returns result. Access members without `it`:

```
val person = Person("Alice", 25).run {
    println("Name: $name, Age: $age")
    "Summary: $name is $age"
}

// Configuring object
val config = Config().run {
    host = "localhost"
    port = 8080
    debug = true
    this  // Returns Config object
}
```

```
// Type conversion
val number = "123".run {
    if (this.isNotEmpty()) toInt() else 0
}
```

with

Like run but called differently. For multiple operations on same object:

```
val person = Person("Bob", 30)

with(person) {
    println("Name: $name")
    println("Age: $age")
    printInfo()  // Method on Person
}

// Doesn't return the object, returns lambda result
val result = with(person) {
    "$name is $age years old"
}
```

apply

Returns the object itself. Used for initialization:

```
val person = Person("Charlie", 35).apply {
    email = "charlie@example.com"
    phone = "555-1234"
}

// Returns Person object, not Unit

// Configuration builder pattern
val config = StringBuilder().apply {
    append("Line 1\n")
    append("Line 2\n")
    append("Line 3")
}.toString()
```

also

Returns the object itself. Often for side effects:

```
val numbers = listOf(1, 2, 3, 4, 5)
    .filter { it > 2 }
```

```
    .also { println("Filtered: $it") }
    .map { it * 2 }
    .also { println("Mapped: $it") }

// also passes object as lambda parameter (it)
Person("David", 28).also {
    println("Created person: ${it.name}")
}
```

Comparison of Scope Functions

Function	Parameter	Returns	Use Case
let	it	Result of lambda	Null-safe, transformations
run	this	Result of lambda	Complex operations, type conversion
with	this	Result of lambda	Multiple operations on object
apply	this	Object itself	Object initialization/configuration
also	it	Object itself	Side effects, logging

---

17. Extension Functions

Basic Extension Functions

```
fun String.countVowels(): Int {
    return this.count { it in "aeiouAEIOU" }
}

println("Hello".countVowels())  // 2

// Without explicit this
fun String.isEmail(): Boolean {
    return contains("@") && contains(".")
}

println("user@example.com".isEmail())  // true
```

Extension Properties

```
val String.firstChar: Char?
    get() = this.firstOrNull()

val String.isAllUpperCase: Boolean
    get() = this == this.uppercase()
```

```
println("Hello".firstChar)      // H
println("HELLO".isAllUpperCase) // true
```

Nullable Receiver

```
fun String?.isNullOrEmpty(): Boolean {
    return this == null || this.isEmpty()
}

val str: String? = null
println(str.isNullOrEmpty())  // true (no safe call needed)

val str2: String? = "Hello"
println(str2.isNullOrEmpty()) // false
```

Extension Functions on Collections

```
fun <T> List<T>.second(): T? {
    return if (this.size >= 2) this[1] else null
}

val numbers = listOf(1, 2, 3)
println(numbers.second())  // 2
```

Member Extension

```
class StringBuilder {
    fun appendLine(text: String) = apply {
        append(text)
        append("\n")
    }
}

// This doesn't actually extend StringBuilder,
// but you can add extensions to classes within a class
```

Scope of Extensions

Extensions are resolved at compile time based on type:

```
fun Any.printType() = println(this::class.simpleName)

val str: String = "Hello"
str.printType()  // String
```

```kotlin
val obj: Any = str
obj.printType()  // Any (compile-time type used)
```

---

18. Infix Functions

Defining Infix Functions

```kotlin
infix fun Int.times(str: String) = str.repeat(this)

// Can be called as infix
3 times "Kotlin"  // "KotlinKotlinKotlin"
// Or traditional way
3.times("Kotlin")
```

Infix with Custom Classes

```kotlin
class Fraction(val numerator: Int, val denominator: Int)

infix fun Int.over(denominator: Int) = Fraction(this, denominator)

val myFraction = 3 over 4
println("${myFraction.numerator}/${myFraction.denominator}")  // 3/4
```

to for Pairs

```kotlin
val pair = 1 to "one"  // Built-in infix function
println(pair.first)   // 1
println(pair.second)  // one

// Creating pairs for maps
val map = mapOf(1 to "one", 2 to "two", 3 to "three")
```

---

19. Data Classes

Basic Data Class

```kotlin
data class Person(val name: String, val age: Int)

val person1 = Person("Alice", 25)
val person2 = Person("Alice", 25)
```

```
// equals and hashCode
println(person1 == person2)  // true (value equality)
println(person1 === person2) // false (reference equality)

// toString
println(person1)  // Person(name=Alice, age=25)

// hashCode
println(person1.hashCode())  // Consistent for equal objects

// copy
val person3 = person1.copy(age = 26)
val person4 = person1.copy()  // Deep copy
```

Destructuring with Data Classes

```
data class Point(val x: Int, val y: Int)

val point = Point(10, 20)
val (x, y) = point
println("$x, $y")  // 10, 20

// In for loops
val points = listOf(Point(0, 0), Point(1, 1), Point(2, 2))
for ((x, y) in points) {
    println("Point: ($x, $y)")
}
```

Custom equals and hashCode

```
data class Custom(val id: Int, val name: String) {
    override fun equals(other: Any?): Boolean {
        if (this === other) return true
        if (other !is Custom) return false
        return id == other.id  // Only compare id
    }

    override fun hashCode(): Int = id.hashCode()
}
```

---

20. Sealed Classes

Sealed Class Hierarchy

```
sealed class Result<out T> {
    data class Success<T>(val data: T) : Result<T>()
    data class Error(val exception: Exception) : Result<Nothing>()
    object Loading : Result<Nothing>()
}

// All subclasses must be in same file/nested
```

Using Sealed Classes with when

```
fun <T> handleResult(result: Result<T>) {
    when (result) {
        is Result.Success -> println("Success: ${result.data}")
        is Result.Error -> println("Error: ${result.exception.message}")
        is Result.Loading -> println("Loading...")
    }
    // No else needed - compiler knows all cases
}

val response: Result<String> = Result.Success("Hello")
handleResult(response)  // Success: Hello
```

---

21. Enumerations

Basic Enum

```
enum class Direction {
    NORTH, SOUTH, EAST, WEST
}

val direction = Direction.NORTH
println(direction)  // NORTH
```

Enum with Properties

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF)
}

println(Color.RED.rgb)  // 16711680
```

Enum with Functions

```
enum class Status {
    PENDING {
        override fun isActive() = false
    },
    RUNNING {
        override fun isActive() = true
    },
    COMPLETED {
        override fun isActive() = false
    };

    abstract fun isActive(): Boolean
}

println(Status.RUNNING.isActive())  // true
```

Enum Iteration

```
enum class Priority {
    LOW, MEDIUM, HIGH
}

for (priority in Priority.values()) {
    println(priority)
}

// By name
val priority = Priority.valueOf("HIGH")  // Priority.HIGH
```

---

22. Object Declarations

Singleton with object

```
object Configuration {
    val apiUrl = "https://api.example.com"
    val apiKey = "secret-key"

    fun reset() {
        // Reset configuration
    }
}

// Usage
```

```
Configuration.apiUrl
Configuration.reset()
```

Object Expression

```
val listener = object : MouseListener {
    override fun onClick() { }
    override fun onDoubleClick() { }
}

// Anonymous object
val person = object {
    val name = "Alice"
    val age = 25
}
```

---

23. Companion Objects

Class Companion Object

```
class MyClass {
    companion object {
        fun create(): MyClass = MyClass()
    }
}

// Call without instance
MyClass.create()
```

Factory Pattern

```
class User private constructor(val name: String, val email: String) {
    companion object {
        fun byName(name: String) = User(name, "unknown@example.com")
        fun byEmail(email: String) = User("Unknown", email)
    }
}

val user1 = User.byName("Alice")
val user2 = User.byEmail("bob@example.com")
```

Companion Object Properties and Methods

```
class Counter {
    companion object {
        var count = 0
        fun reset() { count = 0 }
    }
}

Counter.count = 5
Counter.reset()  // count = 0
```

---

24. Delegation

Class Delegation

```
interface Animal {
    fun sound(): String
    fun move(): String
}

class Dog(val name: String) : Animal {
    override fun sound() = "Woof"
    override fun move() = "Running"
}

class AnimalWrapper(val animal: Animal) : Animal by animal {
    // Delegates all methods to animal
}

val dog = Dog("Buddy")
val wrapper = AnimalWrapper(dog)
println(wrapper.sound())  // Woof
```

Property Delegation

```
class Person(val name: String, age: Int) {
    var age: Int by Delegates.observable(age) { property, oldValue,
newValue ->
        println("${property.name} changed from $oldValue to $newValue")
    }
}

val person = Person("Alice", 25)
person.age = 26  // age changed from 25 to 26

// Delegated with validation
var validated: Int by Delegates.vetoable(0) { property, oldValue, newValue
->
```

```
    newValue >= 0  // Only accept non-negative
}

validated = 10   // OK
validated = -5   // Rejected
```

Custom Property Delegate

```
class Delegate<T>(var value: T) {
    operator fun getValue(thisRef: Any?, property: KProperty<*>): T {
        println("Getting ${property.name} = $value")
        return value
    }

    operator fun setValue(thisRef: Any?, property: KProperty<*>, value: T)
{
        println("Setting ${property.name} = $value")
        this.value = value
    }
}

class MyClass {
    var name: String by Delegate("default")
}

val obj = MyClass()
println(obj.name)     // Getting name = default
obj.name = "Alice"    // Setting name = Alice
```

---

25. Destructuring

Destructuring in Variable Declaration

```
data class Point(val x: Int, val y: Int)

val point = Point(10, 20)
val (x, y) = point
println("$x, $y")  // 10, 20

// Ignoring values
val (x, _) = point
println(x)  // 10
```

Destructuring in Loops

```kotlin
data class Person(val name: String, val age: Int)

val people = listOf(
    Person("Alice", 25),
    Person("Bob", 30)
)

for ((name, age) in people) {
    println("$name: $age")
}
```

Destructuring with Maps

```kotlin
val map = mapOf("a" to 1, "b" to 2, "c" to 3)

for ((key, value) in map) {
    println("$key -> $value")
}

// With entries
for ((key, value) in map.entries) {
    println("$key -> $value")
}
```

Destructuring in Function Parameters

```kotlin
data class User(val id: Int, val name: String)

fun printUser((id, name): User) {
    println("ID: $id, Name: $name")
}

// This syntax doesn't work in Kotlin (unlike some other languages)
// Instead use:
fun printUser(user: User) {
    val (id, name) = user
    println("ID: $id, Name: $name")
}
```

---

26. Generics

Generic Classes

```
class Box<T>(val value: T) {
    fun getValue(): T = value
}

val intBox = Box(42)
val stringBox = Box("Hello")

println(intBox.getValue())     // 42
println(stringBox.getValue())  // Hello
```

Generic Functions

```
fun <T> getFirst(list: List<T>): T? = list.firstOrNull()

val numbers = listOf(1, 2, 3)
val first = getFirst(numbers)  // Type inferred as Int

val strings = listOf("a", "b", "c")
val first2 = getFirst(strings)  // Type inferred as String
```

Multiple Type Parameters

```
class Pair<A, B>(val first: A, val second: B)

val pair = Pair(10, "ten")
println(pair.first)   // 10
println(pair.second)  // ten
```

Bounded Type Parameters

```
// T must be Number or subclass
class NumberBox<T : Number>(val value: T) {
    fun add(other: T): T = value.toDouble() + other.toDouble() as T
}

val intBox = NumberBox(10)
// val stringBox = NumberBox("invalid")  // Error

// Upper bound with & for multiple constraints
fun <T> processData(value: T) where T : Comparable<T>, T : CharSequence {
    // T must be both Comparable and CharSequence
}
```

Generic Variance

Covariance (out)

```
class Box<out T>(val value: T) {
    fun get(): T = value
    // fun set(value: T) {}  // Not allowed
}

val numberBox: Box<Number> = Box(42)
val value: Number = numberBox.get()

// Can assign Box<Int> to Box<Number>
val intBox: Box<Int> = Box(10)
val numberBox2: Box<Number> = intBox  // OK with out
```

Contravariance (in)

```
class Setter<in T> {
    fun set(value: T) { }
    // fun get(): T {}  // Not allowed
}

val setter: Setter<Int> = Setter()
val superSetter: Setter<Number> = setter  // OK with in
superSetter.set(42)  // OK
```

---

27. Type Projections

Use-site Variance

```
// Covariance
fun printList(list: List<out Any>) {
    for (item in list) {
        println(item)
    }
}

val intList: List<Int> = listOf(1, 2, 3)
printList(intList)  // OK

// Contravariance
fun fillList(list: MutableList<in Int>) {
    list.add(1)
    list.add(2)
}
```

```
val numberList: MutableList<Number> = mutableListOf()
fillList(numberList)  // OK
```

Type Erasure

```
// At runtime, generic type information is erased
val intList: List<Int> = listOf(1, 2, 3)
val stringList: List<String> = listOf("a", "b")

println(intList is List<Int>)    // Error: type is erased
println(intList is List<*>)      // OK: works

// Reified generics (only with inline functions)
inline fun <reified T> isInstance(obj: Any) = obj is T
```

---

28. Coroutines

Basic Coroutine Concepts

Coroutines are lightweight threads that can be suspended and resumed:

```
// Requires: implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-core'

import kotlinx.coroutines.*

// Launch a coroutine
fun example() {
    GlobalScope.launch {
        println("Hello from coroutine")
        delay(1000)  // Non-blocking delay
        println("After delay")
    }
    println("Main thread continues")
}

// Using runBlocking (blocks until complete)
fun example2() {
    runBlocking {
        launch {
            delay(100)
            println("Coroutine done")
        }
        println("Main continues")
    }
    println("After runBlocking")
}
```

Async and Await

```
fun asyncExample() {
    runBlocking {
        val deferred = async {
            delay(1000)
            "Result"
        }

        val result = deferred.await()
        println(result)
    }
}

// Multiple parallel operations
fun multipleAsync() {
    runBlocking {
        val deferred1 = async { delay(1000); 1 }
        val deferred2 = async { delay(1000); 2 }

        val result1 = deferred1.await()
        val result2 = deferred2.await()

        println(result1 + result2)  // 3
    }
}
```

Scope and Context

```
// CoroutineScope manages lifecycle
fun scopeExample() {
    runBlocking {
        launch {
            // Launched in this scope
            println("Child coroutine")
        }
        println("Parent continues")
    }  // Waits for children to complete
}

// withContext switches context
fun contextExample() {
    runBlocking {
        val result = withContext(Dispatchers.Default) {
            // Run in Default dispatcher
            computeExpensive()
        }
        println(result)
```

```
    }
}
```

Dispatchers

```
// Main dispatcher - UI thread
launch(Dispatchers.Main) { }

// Default dispatcher - CPU-intensive work
launch(Dispatchers.Default) { }

// IO dispatcher - I/O operations
launch(Dispatchers.IO) { }

// Unconfined dispatcher
launch(Dispatchers.Unconfined) { }

// Custom dispatcher
val singleThread = Dispatchers.Unconfined
```

---

29. Exception Handling

Try-Catch-Finally

```
try {
    val result = 10 / 0
} catch (e: ArithmeticException) {
    println("Error: Division by zero")
} catch (e: Exception) {
    println("General error: ${e.message}")
} finally {
    println("Finally block always executes")
}
```

Try as Expression

```
val number = try {
    "42".toInt()
} catch (e: NumberFormatException) {
    0
}
println(number)  // 42

val result = try {
    "abc".toInt()
```

```
} catch (e: NumberFormatException) {
    -1
} finally {
    println("Cleanup")
}
println(result)  // -1
```

Throwing Exceptions

```
fun validateAge(age: Int) {
    if (age < 0) {
        throw IllegalArgumentException("Age cannot be negative")
    }
}

fun throwError(): Nothing {
    throw IllegalStateException("This always throws")
}
```

Custom Exceptions

```
class InvalidUserException(message: String) : Exception(message)

class UserService {
    fun getUser(id: Int) {
        if (id < 0) {
            throw InvalidUserException("Invalid user ID: $id")
        }
    }
}

try {
    UserService().getUser(-1)
} catch (e: InvalidUserException) {
    println("Invalid user: ${e.message}")
}
```

runCatching

```
val result = runCatching {
    "42".toInt()
}.onSuccess { value ->
    println("Success: $value")
}.onFailure { exception ->
    println("Failure: ${exception.message}")
}
```

```
// Getting result
val value = result.getOrNull()  // 42 or null
val value2 = result.getOrElse { -1 }  // 42 or -1
```

---

30. Package and Imports

Package Declaration

```
// Must be first line (except for file annotations and package-level
comments)
package com.example.myapp

class MyClass
fun myFunction() { }
```

Imports

```
// Specific import
import java.util.Scanner

// Import multiple from same package
import java.util.Scanner
import java.util.ArrayList

// Wildcard import
import java.util.*

// Aliasing imports
import java.util.Scanner as JavaScanner
import com.example.Scanner as MyScanner

val scanner1 = JavaScanner(System.`in`)
val scanner2 = MyScanner()

// Importing functions
import kotlin.math.max
import kotlin.math.min

val result = max(5, 10)
```

Default Imports

These packages are imported by default:

```
kotlin.*
kotlin.annotation.*
kotlin.collections.*
kotlin.comparisons.*
kotlin.io.*
kotlin.ranges.*
kotlin.sequences.*
kotlin.text.*
```

---

31. Visibility Modifiers

Visibility in Classes and Objects

```
class MyClass {
    public val publicProperty = "Public"    // Visible everywhere
    private val privateProperty = "Private" // Only in this class
    protected val protectedProperty = "Protected" // In subclasses
    internal val internalProperty = "Internal"    // In same module
}

// Default is public
class Another {
    val implicitlyPublic = "Public"
}
```

Function Visibility

```
class Service {
    public fun publicMethod() { }    // Visible everywhere
    private fun privateMethod() { }  // Only in this class
    protected fun protectedMethod() { } // In subclasses
    internal fun internalMethod() { }   // In same module
}
```

Top-level Visibility

```
// File: MyUtils.kt
package com.example

public fun publicFunction() { }      // Visible in all modules
private fun privateFunction() { }    // Only in this file
internal fun internalFunction() { }  // Only in this module

public class PublicClass { }
```

```
private class PrivateClass { }       // Only in this file
internal class InternalClass { }     // Only in this module
```

Getters and Setters Visibility

```
class Person {
    var age: Int = 0
        get() = field
        private set(value) {
            if (value >= 0) field = value
        }

    // Setter is private, getter is public
    var salary: Double = 0.0
        private set
}

val person = Person()
person.age = 25        // OK
person.salary = 50000  // Error: setter is private
val age = person.age   // OK
```

---

32. Best Practices

Use val Over var

```
// Prefer immutability
val list = mutableListOf(1, 2, 3)  // Reference is immutable
list.add(4)  // But contents can be modified

// Over
var list2 = listOf(1, 2, 3)  // Reassignable
```

Null Safety

```
// Prefer this
val length = name?.length ?: 0

// Over
val length2: Int
if (name != null) {
    length2 = name.length
} else {
    length2 = 0
}
```

Use Early Returns

```
fun processUser(user: User?): String {
    user ?: return "No user"

    if (!user.isActive) return "Inactive"

    return "Processing ${user.name}"
}

// Over
fun processUser2(user: User?): String {
    return if (user != null) {
        if (user.isActive) {
            "Processing ${user.name}"
        } else {
            "Inactive"
        }
    } else {
        "No user"
    }
}
```

Use Scope Functions Appropriately

```
// For null-safe operations with transformations
val result = userName?.let { it.uppercase() }

// For configuration/initialization
val config = Config().apply {
    apiUrl = "https://api.example.com"
    timeout = 5000
}

// For object inspection during chain
val numbers = (1..10)
    .filter { it % 2 == 0 }
    .also { println("Filtered: $it") }
    .map { it * 2 }
```

Prefer Data Classes for Simple Data Holders

```
// Good: Simple data holder
data class User(val id: Int, val name: String, val email: String)

// Over: Regular class with boilerplate
```

```
class User(val id: Int, val name: String, val email: String) {
    override fun equals(other: Any?) = // ...
    override fun hashCode() = // ...
    override fun toString() = // ...
}
```

Use Sealed Classes for Type-Safe Enums

```
// Type-safe alternative to enums
sealed class Result<out T> {
    data class Success<T>(val data: T) : Result<T>()
    data class Error(val exception: Exception) : Result<Nothing>()
    object Loading : Result<Nothing>()
}

// Exhaustive when
fun handle(result: Result<String>) {
    when (result) {
        is Result.Success -> println(result.data)
        is Result.Error -> println(result.exception)
        is Result.Loading -> println("Loading...")
    }
}
```

Extension Functions Over Utility Classes

```
// Good: Extension function
fun String.toTitleCase() = this.split(" ")
    .joinToString(" ") { it.capitalize() }

println("hello world".toTitleCase())

// Over: Utility class
object StringUtils {
    fun toTitleCase(str: String) = // ...
}

println(StringUtils.toTitleCase("hello world"))
```

Avoid!! Operator

```
// Bad: Can cause NPE
val name = user!!.name

// Better: Safe alternatives
val name = user?.name ?: "Unknown"
```

```
val name2 = user?.name ?: run {
    handleMissingUser()
    "Unknown"
}
```

Use Naming Conventions

```
// Constants
const val MAX_SIZE = 100

// Variables and functions (camelCase)
val userName = "Alice"
fun calculateSum() { }

// Classes and interfaces (PascalCase)
class User
interface Animal

// Enum values (UPPERCASE)
enum class Priority {
    HIGH, MEDIUM, LOW
}
```

Documentation Comments

```
/**
 * Calculates the sum of two numbers.
 *
 * @param a First number
 * @param b Second number
 * @return The sum of a and b
 * @throws IllegalArgumentException if either parameter is negative
 */
fun sum(a: Int, b: Int): Int {
    if (a < 0 || b < 0) throw IllegalArgumentException()
    return a + b
}
```

Unit Testing

```
// With JUnit and Kotlin test libraries
import org.junit.Test
import org.junit.Assert.*

class CalculatorTest {
    @Test
```

```
    fun `test addition`() {
        val result = 2 + 2
        assertEquals(4, result)
    }

    @Test
    fun `test division by zero throws exception`() {
        assertThrows(ArithmeticException::class.java) {
            val result = 10 / 0
        }
    }
}
```

Performance Considerations

```
// Prefer sequences for large collections with multiple operations
val result = (1..1_000_000)
    .asSequence()  // Lazy evaluation
    .filter { it % 2 == 0 }
    .map { it * 2 }
    .take(10)
    .toList()

// Over eager evaluation
val result2 = (1..1_000_000)
    .filter { it % 2 == 0 }  // Processes all elements
    .map { it * 2 }          // Processes all elements
    .take(10)                // Only takes first 10
```